

Google ADK Integration Walkthrough

Overview

The backend has been successfully upgraded to feature a **Google ADK Multi-Agent Workflow**, running parallel to our legacy architecture. We successfully encapsulated the battle-tested, deterministic models into isolated tool functions, allowing Google ADK Agent instances to process the input through a highly structured pipeline.

By enforcing strict tool executions, we completely bypassed any unwanted LLM hallucination for critical steps like visual disease detection while maintaining full ADK compliance.

Architecture & Tool Wrappers

To integrate with Google ADK, we adopted an ADK Agent Tool Wrapper Legacy Agent pattern.

1. Tool Wrappers ([tools.py](file:///d:/agricare/agents/adk_workflow/tools.py))

- We instantiated the underlying logic blocks (DiseaseDetectionAgent , SeverityAgent , AdvisoryAgent) securely within standard python functions (detect_disease_tool , analyze_severity_tool , generate_advice_tool).
- These tools handle exceptions internally and conform to the ADK standard signature.

2. ADK Coordinator ([adk_coordinator.py]

(file:///d:/agricare/agents/adk_workflow/adk_coordinator.py))

- Creates three isolated ADK Agent instances (DiseaseAgent , SeverityAgent , AdvisoryAgent), mapping the specific tools to their respective intelligence nodes.
- Orchestrates the state between the agents deterministically without relying on Gemini to “guess” whether to execute the image recognition tool.
- Enforces a **mandatory fallback policy**: if the Google ADK library is missing or GEMINI_API_KEY is not present, the ADKCoordinatorAgent seamlessly passes the task to the Legacy CoordinatorAgent .

Architecture Diagram

```
graph TD
    UI[Streamlit UI]
    subgraph Orchestration_Engine_Selector [Orchestration Engine Selector]
        toggle{Workflow Mode}
    end

    UI --> toggle
    toggle -- "Legacy Coordinator" --> Legacy[Legacy Coordinator Agent]
    toggle -- "Google ADK Workflow" --> ADK[ADK Coordinator Agent]

    subgraph ADK_Multi-Agent_Workflow [ADK Multi-Agent Workflow]
        ADK --> D_Agent[DiseaseAgent]
        D_Agent -- Forces Call --> D_Tool[detect_disease_tool]
        D_Tool -. " " .-> Legacy_D[DiseaseDetectionAgent]

        D_Tool --> S_Agent[SeverityAgent]
        S_Agent --> S_Tool[analyze_severity_tool]
        S_Tool -. " " .-> Legacy_S[SeverityAgent]

        S_Tool --> A_Agent[AdvisoryAgent]
        A_Agent --> A_Tool[generate_advice_tool]
        A_Tool -. " " .-> Legacy_A[AdvisoryAgent]
    end

    Legacy_A -. " " .-> ADK
    ADK --> |Unified JSON| UI
    Legacy --> |Unified JSON| UI
    ADK -. " " .-> |Fallback if API Fails| Legacy
```

Demo Instructions

1. Ensure the Streamlit server is running (`venv\Scripts\streamlit run app/main.py`).
2. Open the UI to the **Disease Detection** page.
3. In the left sidebar, locate the **Workflow Settings** section.
4. Toggle the radio button to **Google ADK Workflow**.
5. Upload an image and click **Run Prediction**.

6. At the bottom of the page in the **Pipeline Diagnostics** section, you will see a `GOOGLE_ADK` badge, proving the system ran entirely through the ADK multi-agent layer.
7. To test the mandatory fallback, simply remove your `GEMINI_API_KEY` from your environment. Running the prediction again will gracefully execute the legacy pipeline without crashing the app, alerting you via the diagnostics footer.